Fault-Tolerant Control of a Quadrotor After Complete Rotor Loss

Residual Koopman MPC and Certified Fault-Detection-Delay Budgets

EE650 Basics of Modern Control Systems: Progress Report

Pranshu Kumar (220799) Mudita Jain (220675)
Prashant Gupta (220803) Aarush Singh Kushwaha (230019)

October 2026

Abstract

When one rotor of a quadrotor fails completely, the vehicle can no longer hover level. Its only steady state is a constant spin about a tilted body axis, the *relaxed hover* of Mueller and D'Andrea. This project studies how to recover a quadrotor into this state reliably, in simulation. We use the known physics of the spinning vehicle as a backbone and learn only its mismatch with a Koopman (EDMDc) model inside model predictive control. We also ask how long fault detection may take before recovery can no longer be guaranteed. This report covers the first phase of the work. We built a simulator of the vehicle used by Mueller and D'Andrea and validated it against three published papers: it reproduces their two-rotor and three-rotor-loss equilibria to every published digit, five hover solutions of their detailed model within 4%, and the spin-up time of their flight within about 4%. We analysed controllability and observability and designed LQR baselines. For horizons of 100 ms and longer, the residual Koopman model is the most accurate predictor on the training plant, and it needs less data than a full-state Koopman model; a residual-corrected MPC reduces a steady altitude error caused by model mismatch by one third. The learned models do not yet generalize to an unseen plant. In our simulation, the published recovery strategy recovers safely with probability above 0.9 only for detection delays up to about 12 ms, and below 0.5 beyond about 19 ms, while fast detectors need 30 to 130 ms. A first saturation-aware LMI certificate covers 22% of the way from the equilibrium to the state right after a failure in hover. We close with the limitations and the plan for the remaining weeks.

Contents

1 Introduction **3**

2 Related work **3**

3 System model **4**
3.1 Equations of motion 4
3.2 Parameters and truth plants 5

4 Model validation **5**

5 Linear analysis and baseline control **6**
5.1 Linear models 6
5.2 Controllability and observability 6
5.3 LQR baselines 6

6 Residual Koopman models and MPC **7**
6.1 Predictors 7
6.2 Prediction results 8
6.3 MPC in closed loop 9

7 Recovery under fault-detection delay **10**
7.1 The undetected phase 10
7.2 Effect of detection delay 10
7.3 A first saturation-aware certificate 11

8 Limitations **12**

9 Plan for the remaining weeks **13**

10 Conclusion **13**

A Software and reproducibility **14**

1 Introduction

Quadrotors have no actuator redundancy. If one rotor stops, the remaining three cannot produce zero yaw torque while carrying the weight, so the vehicle starts to spin. Mueller and D'Andrea showed that the vehicle can still be controlled by giving up the yaw angle. The vehicle spins at a constant rate about a body-fixed axis, and it translates by tilting this axis [1, 2]. In body-fixed coordinates this *relaxed hover* is a fixed point, so linear time-invariant tools apply. They stabilised it with LQR and demonstrated it in flight.

Recovering a real vehicle into this state raises two connected problems. The first is model accuracy. The physics model captures the dominant spinning motion but misses effects such as thrust loss at high spin rate, parameter errors and motor dynamics. A natural remedy is to keep the physics model and learn only its mismatch, then use the corrected model inside model predictive control (MPC). Koopman-operator methods suit this, because they give linear predictors that keep the MPC problem convex [3, 4]. Residual Koopman models of this kind exist for healthy quadrotors and other robots [5–7], but we did not find them applied to the spinning regime after complete rotor loss.

The second problem is timing. A fault detector needs time to notice the failure. Fast detectors for propeller loss report delays of 30 to 130 ms [8]. During that time the normal controller keeps flying a vehicle that has lost a rotor, and the failure-mode controller must then capture the vehicle from wherever it has drifted. Most recovery methods with formal guarantees assume that detection is instantaneous [9, 10]. Başak and Prempain prove stability for a switched linear model that includes a fixed detection delay [11, 12], and passive schemes avoid the detection step altogether [13, 14]. We did not find a certified *maximum* tolerable detection delay that accounts for thrust saturation and the drift of the undetected phase.

This project addresses both problems together. Its central question is whether a physics-informed residual Koopman model gives a better predictor for MPC than the physics model alone or a full-state Koopman model. Its second aim is a certified bound $\tau^\star$ on the tolerable detection delay. The two are linked: every controller that captures the vehicle after detection, whether LQR, analytical MPC or residual-Koopman MPC, can be compared by the delay it tolerates.

Work reported here. (i) A simulator of the Mueller–D'Andrea vehicle, validated against three papers (Section 4). (ii) Linear analysis and LQR baselines (Section 5). (iii) Residual Koopman predictors and MPC (Section 6). (iv) The effect of detection delay and a first LMI certificate (Section 7). Section 8 lists the limitations and Section 9 the remaining work. Everything is in simulation; no hardware is used.

2 Related work

Complete rotor loss. Freddi, Lanzon and Longhi gave up yaw control and stabilised a horizontal spin with feedback linearization and H_∞ loop shaping [15, 16]. Mueller and D'Andrea derived the relaxed hover for one, two and three lost propellers, proved controllability of its linearization and validated it in flight [1, 2]. Stephan et al. designed an LPV controller scheduled on the yaw rate to cover the transient after the loss [17], and Khattab et al. combined an LPV reduced-attitude model with sliding-mode control [18]. Sun et al. used incremental nonlinear dynamic inversion at high speed [19] and a quadratic-programming allocation for recovery from large upsets [20]. Nan et al. used nonlinear MPC with per-rotor thrust limits [21], and Sun, Zhang and Lang learned an unknown complete rotor failure inside nonlinear MPC [22]. Ke, Cai and Quan proposed a passive controller that tolerates one to three rotor failures without detection [13].

Guarantees and detection delay. Simha et al. give an analytic region of attraction for a geometric controller, with the fault assumed to be known from the start [9]. Al Lawati and Akhtar prove path invariance under quasi-static feedback and state that they assume a reliable fault-detection module and an instantaneous transition [10]. Başak and Prempain model the recovery as a switched system and use multiple Lyapunov functions and dwell-time conditions that include a fixed detection delay; their analysis uses a linearized model without thrust saturation and does not compute the largest tolerable delay [11, 12]. Qi et al. compute a recoverable set at the relaxed hover with sum-of-squares programming and rotor-speed limits [23]. Reported tolerances are empirical: 180 ms for a variable-pitch vehicle [24], and recovery with about 0.1 to 0.18 s of detection delay for nonlinear controllers in simulation and flight [14, 25]. Fast detectors report 30 to 130 ms [8] and about 45 ms [26]. Set-theoretic fault-tolerant control uses invariant and reachable sets for detection and reconfiguration in general [27], and detection delay has been analysed for general fault-tolerant systems [28]. A motion planner based on backward reachability exists for the failed quadrotor after the fault is handled [29].

Koopman models and residual learning. EDMD approximates the Koopman operator on a dictionary of observables [4]. With inputs it gives linear predictors for MPC [3, 30]. Koopman MPC exists for healthy quadrotors [31, 32]. Koopman-based fault-tolerant MPC and fault detection address partial loss of effectiveness, with all rotors still working [33, 34]. Physics models corrected by learned Koopman residuals exist for soft robots, ground vehicles, quadrupeds and healthy quadrotors [5–7, 35, 36]. Todde et al. evaluate the residual outside the quadratic program and inject it as a measured disturbance, which keeps the problem convex, and list formal robustness guarantees as future work [5]. LMI-based controller design with bounded Koopman approximation errors is developed in SafEDMD [37]. We did not find residual Koopman models applied to the complete-loss, spinning-equilibrium regime, nor a certified maximum tolerable detection delay under thrust saturation. We state this as the outcome of our search, not as proof that no such work exists.

3 System model

3.1 Equations of motion

We follow the model of [1]. The body frame has z along the thrust direction. Rotor 1 is on the $+x$ arm, rotor 2 on $+y$, rotor 3 on $-x$ and rotor 4 on $-y$, all at distance l. Rotors 1 and 3 spin with negative speed, and rotors 2 and 4 with positive speed. With thrusts $f_i = \kappa_f \omega_i^2$, position $\boldsymbol{d}$, attitude R and body rates $\boldsymbol{\omega} = (p, q, r)$,

$$m\ddot{\boldsymbol{d}} = R\,\boldsymbol{e}_3 \textstyle\sum_i f_i - mg\,\boldsymbol{e}_3, \tag{1}$$

$$I^B\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (I^T\boldsymbol{\omega} + I^P_{zz}\,\omega_\Sigma\,\boldsymbol{e}_3), \qquad \omega_\Sigma = \textstyle\sum_i \omega_i, \tag{2}$$

$$\boldsymbol{\tau} = \big(\, l(f_2 - f_4),\; l(f_3 - f_1),\; \kappa_\tau(f_1 - f_2 + f_3 - f_4) - \gamma r \,\big).$$

Here I^B is the body inertia, $I^T = I^B + 4I^P$ includes the propellers, and γ is a linear yaw-drag coefficient. As in the paper, the term $I^P\dot{\boldsymbol{\omega}}$ is neglected. Each motor follows its command with a first-order lag, $\dot{f}_i = (\mathrm{sat}(u_i) - f_i)/\sigma$. A running motor produces $f_i \in [f_{\min}, f_{\max}]$, and a command $u_i \le 0$ stops it. A failed rotor produces no thrust. The attitude is stored as a quaternion and integrated with a fixed-step fourth-order Runge–Kutta method at 0.5 to 1 ms. Controllers run at 1 to 5 ms with a zero-order hold.

Reduced attitude. After the failure we control only the direction $\boldsymbol{n} = R^\top \boldsymbol{e}_3$, which is the inertial "up" direction seen from the body. It obeys

$$\dot{\boldsymbol{n}} = -\boldsymbol{\omega} \times \boldsymbol{n}. \tag{3}$$

Relaxed hover. An equilibrium has constant $\boldsymbol{\omega}$ and $\boldsymbol{n}$ in the body frame. This requires $\dot{\boldsymbol{\omega}} = 0$ in (2), $\boldsymbol{n} \parallel \boldsymbol{\omega}$ (so that $\dot{\boldsymbol{n}} = 0$), and $f_\Sigma n_z = mg$ for zero mean vertical acceleration. With rotor 4 lost, $f_1 = f_3$ and $f_2 = \rho f_1$, the rate equations have a closed form: $p = 0$, $r = \kappa_\tau(f_1 - f_2 + f_3)/\gamma$ and $q = l f_2/((I^T_{zz} - I^T_{xx})r + I^P_{zz}\omega_\Sigma)$. A one-dimensional root search on f_1 then gives the equilibrium. For the design ratio $\rho = 0.5$, $\boldsymbol{f} = (2.04, 1.02, 2.04, 0)\,\mathrm{N}$, the vehicle spins at 19.6 rad/s about its axis $\bar{\boldsymbol{n}}$, and this axis is tilted by 16° from the body z axis.

3.2 Parameters and truth plants

All parameters are those of the vehicle in [1, Sec. V-A] (Table 1). For the learning experiments we also use *truth plants* that contain effects the nominal model lacks. The first is a loss of thrust efficiency at high body rate, $\eta = 1 - k\|\boldsymbol{\omega}\|^2$; the paper reports less thrust than expected while spinning. The others are an inertia error and a different motor time constant. Family A is used for training. Family B uses different magnitudes and is held out.

Table 1: Vehicle parameters [1] and truth-plant families.

Quantity	Symbol	Value
Mass, arm length	m, l	0.50 kg, 0.17 m
Total inertia	I^T_{xx}, I^T_{zz}	3.2, $5.5 \times 10^{-3}\,\mathrm{kg\,m^2}$
Propeller inertia	I^P_{zz}	$1.5 \times 10^{-5}\,\mathrm{kg\,m^2}$
Thrust coefficient	κ_f	$6.41 \times 10^{-6}\,\mathrm{N\,s^2/rad^2}$
Torque-to-thrust ratio	κ_τ	0.0169 m
Yaw drag	γ	$2.75 \times 10^{-3}\,\mathrm{N\,m\,s/rad}$
Thrust range, motor lag	$[f_{\min}, f_{\max}]$, σ	[0.2, 3.8] N, 15 ms
Family A (training)		$k = 1.5 \times 10^{-4}\,\mathrm{s^2/rad^2}$, $I_{xx} \times 1.08$, $I_{zz} \times 0.95$, $\sigma = 20$ ms
Family B (held out)		$k = 2.5 \times 10^{-4}\,\mathrm{s^2/rad^2}$, $I_{xx} \times 0.94$, $I_{zz} \times 1.06$, $\sigma = 12$ ms

4 Model validation

We validated the simulator in two ways. First, automated tests compare it with hand-derived results: free fall; thrust direction; closed-form torque-free precession; conservation of inertial angular momentum and kinetic energy (to 10^{-8}); reduced-attitude kinematics against the matrix exponential; the first-order yaw spin-up; the motor-lag step response; the fourth-order convergence of the integrator; and an independent transcription of the paper's rate equations. Second, we compared it with published numbers from three papers (Table 2).

Three points need comment. In the one-rotor case, the paper's published values do not satisfy its own roll-rate equation: they leave a residual torque of $-0.011\,\mathrm{N\,m}$. Its other two equilibria match exactly with the same parameters, so we treat this as an inconsistency in that one published case. The IJRR model computes thrust from the propeller speed relative to the air, $\Omega_i + r$, and uses quadratic drag. We implemented it separately; with these effects switched off, it reduces exactly to our simulator, which the tests confirm. As a sensitivity check, an inertia $I_{xx} = 2.75 \times 10^{-3}\,\mathrm{kg\,m^2}$ (the upper end of the rounding interval of the published value) brings all five cases within 0.7%; because this value was chosen to fit the same cases, it is not an independent validation. Finally, the spin-up time depends strongly on the thrust of the two remaining rotors: at the thrust used in flight the model is within about 4% of the flight, while at $mg/2$ it would take 0.81 s. The delay results of Section 7 are therefore reported for both thrusts. The papers' models over-predict the steady spin rate in flight by about 20% [2].

Table 2: Comparison with published results. Full tables: `results/tables/validation_report.md`.

Source	What was compared	Result
ICRA 2014 [1]	Two-rotor-loss equilibrium, eqs. (57) to (59)	$f = 2.453\,\mathrm{N}$, $\omega_1 = -618.6\,\mathrm{rad/s}$, $r = 30.14\,\mathrm{rad/s}$ against 2.45, −619, 30.1: all digits (≤0.14%)
ICRA 2014	Three-rotor-loss equilibrium, eqs. (62) to (64)	$\boldsymbol{\omega} = (14.68, 0, 32.99)$ against (14.7, 0, 33.0), $\boldsymbol{n} = (0.407, 0, 0.914)$ against (0.41, 0, 0.91): all digits
ICRA 2014	One-rotor-loss equilibrium, $\rho = 0.5$	f, r, n_z within 0.5%; q and n_y 6% lower (see text)
ICRA 2014	Fig. 3: minima over ρ	Values within 2%, locations within about 3%
ICRA 2014	Coupling constant $\bar{a}$, eq. (28); controllability loss, eq. (39)	Exact; controllability rank drops from 7 to 6, with p uncontrollable, exactly when $\bar{a} = 0$
IJRR 2016 [2]	Five hover solutions (no failure; one; two opposing; two adjacent; three rotors lost), independent implementation of their model	Body-rate vector within 4% with the published parameters
Sun et al. [19]	Bebop 2, two-rotor loss, eq. (60)	Equal to the paper's formula; the paper's quoted values are 1.5% below its own formula
Flight data [1]	Time to spin up to 10 rad/s on two rotors at the flight thrust (2.95 N, Fig. 4)	0.647 s against 0.62 s in flight (4%)

5 Linear analysis and baseline control

5.1 Linear models

At hover the healthy vehicle has 12 states and four inputs. After the failure we use $\boldsymbol{s} = (p, q, r, n_x, n_y, z, v_z)$ with inputs (f_1, f_2, f_3). These equations are autonomous in body coordinates, so their linearization about the relaxed hover is time invariant. Both models are Jacobians of the exact equations, checked against finite differences. The failed model has open-loop eigenvalues $\pm 10.10j$ (gyroscopic coupling $\bar{a}$), $\pm 19.56j$ (equal to $\|\bar{\boldsymbol{\omega}}\|$), -0.51 (yaw drag) and a double integrator (altitude). The exact coefficient of n_y in $\dot{n}_x$ is $\bar{r} + \bar{q}\bar{n}_y/\bar{n}_z = 20.3$; the simplified matrix in [1] uses $\bar{r} = 18.8$.

5.2 Controllability and observability

Both models are controllable: the controllability matrix has full rank and every mode passes the PBH test. Table 3 lists observability for four sensor sets of the failed vehicle. To make the observability Gramians comparable, each output is divided by an assumed sensor noise level (gyro 0.005 rad/s, tilt estimate 0.01, barometer 0.1 m). Gyros alone see neither tilt nor altitude. The relaxed hover is tilted ($\bar{n}_y \neq 0$), so the vertical acceleration depends on tilt to first order, and gyro plus barometer makes the full state observable. For these noise levels, however, the smallest eigenvalue of the 1 s Gramian is about 2,700 times smaller than with a direct tilt measurement. This is a structural remark, not a practical estimator; practical estimators for the spinning vehicle use the accelerometer and vision [38, 39]. For the healthy vehicle, gyro plus position leaves the heading unobservable, as expected.

5.3 LQR baselines

The healthy LQR uses Bryson weights. The failure-mode LQR uses the weights of [1] (1 on p, q; 20 on n_x, n_y; $1\,\mathrm{N}^{-2}$ on the inputs). We add the motor states and altitude; the motor states are

Table 3: Observability of the failed vehicle's linear model (7 states). Gramian computed with noise-normalized outputs.

Sensors	Rank	Unobservable	$\lambda_{\min}$ of $W_o(1\,\mathrm{s})$
Gyro (p, q, r)	3/7	n_x, n_y, z, v_z	0
Gyro + tilt estimate (n_x, n_y)	5/7	z, v_z	0
Gyro + barometer (z)	7/7	none	2.4×10^{-3}
Gyro + tilt + barometer	7/7	none	6.6

not measured, so the controller reconstructs them by filtering its own commands. Table 4 gives the closed-loop properties. On the nonlinear model with motor lag and thrust limits, the failure LQR returns from a perturbed start (Fig. 1); the tilt settles within 0.01 in 2.1 s, the altitude within 5 cm in 1.8 s, and the yaw rate within 0.5 rad/s in 2.5 s. The least-damped mode is the oscillation at about $\|\bar{\boldsymbol{\omega}}\|$ ($-3.80 \pm 19.68j$, damping ratio 0.19). The input-delay margin of the failure LQR is only 18 ms, against 40 ms for the healthy one.

Table 4: LQR designs. The delay margin is the largest pure input delay that keeps the linear closed loop stable.

Controller	Closed-loop poles (slowest first)	Delay margin
Healthy LQR (12 states)	$-1.72 \pm 1.02j$, -1.89, $-1.94 \pm 1.56j$	40 ms
Failure LQR (7 + 3 motor states)	-1.09, $-2.01 \pm 1.04j$, $-3.80 \pm 19.68j$	18 ms

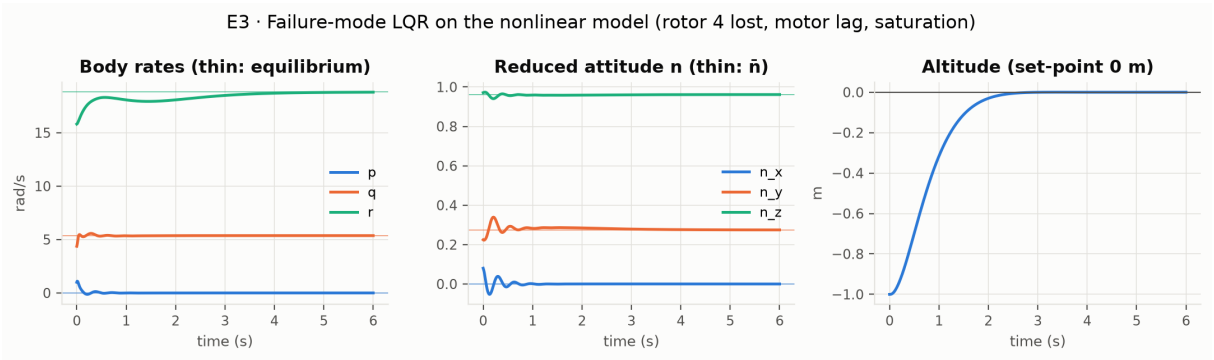


Figure 1: Failure-mode LQR on the nonlinear model (rotor 4 lost, motor lag, thrust limits), starting from a perturbed relaxed hover 1 m below the set-point.

6 Residual Koopman models and MPC

6.1 Predictors

All predictors use the same data. These are 40 closed-loop episodes of 4 s on truth family A, sampled at $T_s = 5$ ms, with the failure LQR and random excitation added to the commands. The measured state is $\boldsymbol{x} = (p, q, r, n_x, n_y, z, v_z)$ (deviations from the relaxed hover). The thrusts the controller knows are the nominal first-order filter $\hat{\boldsymbol{f}}$ of its own commands.

- **Analytical.** The Mueller–D'Andrea linear model with motor states, discretized with a zero-order hold: $\boldsymbol{s}_{k+1} = A_d \boldsymbol{s}_k + B_d \boldsymbol{u}_k$, with $\boldsymbol{s} = (\boldsymbol{x}, \hat{\boldsymbol{f}})$.
- **Full-state EDMDc** [3, 4]. Lifted state $\boldsymbol{z} = \psi(\boldsymbol{\xi})$ with $\boldsymbol{\xi} = (p, q, r, n_x, n_y, v_z, \hat{\boldsymbol{f}})$. The dictionary ψ holds a constant, the standardized $\boldsymbol{\xi}$ and all its degree-2 monomials (55 functions). We fit $\boldsymbol{z}_{k+1} \approx A\boldsymbol{z}_k + B\boldsymbol{u}_k$ by ridge-regularized least squares.

- **Residual EDMDc (proposed).** The analytical prediction plus a learned one-step correction,

$$\boldsymbol{x}_{k+1} = [A_d \boldsymbol{s}_k + B_d \boldsymbol{u}_k]_{1:7} + \mathrm{clip}\left(W\psi(\boldsymbol{\xi}_k, \boldsymbol{u}_k),\, \pm \boldsymbol{d}_{\max}\right), \tag{4}$$

where W is fitted by ridge regression on the one-step residuals and $\boldsymbol{d}_{\max}$ is the 99.5th percentile of the training residuals.

6.2 Prediction results

Figure 2 and Table 5 show the open-loop multi-step error on held-out episodes. On family A the residual model is the most accurate predictor at every horizon from 100 ms on; at shorter horizons the full-state model is slightly better. The analytical model drifts: its undamped modes and the unmodelled thrust loss make it worse than the trivial "no change" predictor. Full-state EDMDc has 4 of its 55 eigenvalues outside the unit circle (spectral radius 1.0070) and falls behind the no-change predictor by 2 s, as our proposal predicted. At 2 s the residual model also exceeds the no-change error, so long open-loop horizons need feedback.

Table 5: Normalized multi-step RMSE (mean over the 7 states), models trained on family A.

	Test family A			Test family B (held out)		
Predictor	0.1 s	1 s	2 s	0.1 s	1 s	2 s
Analytical	3.17	7.09	7.93	4.31	7.94	6.95
Full-state EDMDc	0.17	0.91	3.45	6.09	26.2	131
Residual EDMDc	**0.15**	**0.64**	2.71	5.98	19.9	24.0
No change	2.42	2.38	2.51	2.51	2.44	2.41



Figure 2: Multi-step prediction error. Top: models trained on family A. Bottom: models trained on randomized plants. Left: test on family A; right: test on the held-out family B.

Data efficiency. At a 0.5 s horizon, the residual model trained on 10 episodes (error 0.42) is more accurate than the full-state model trained on 40 (0.48) (Fig. 3). This matters because

data from failed flights is scarce.

Generalization. On the held-out family B, both models trained on family A are worse than the analytical model (Table 5). Training on randomized plants, with one random plant per episode, trades accuracy: it improves the residual model on family B up to about 100 ms (error 3.68 against 4.31 for the analytical model) but makes both learned models worse on family A (Fig. 2, bottom). The likely reason is that the motor time constant and thrust loss of the current plant cannot be inferred from a single state sample, so a static regression averages over them. This motivates online adaptation of W, which is linear in the data.

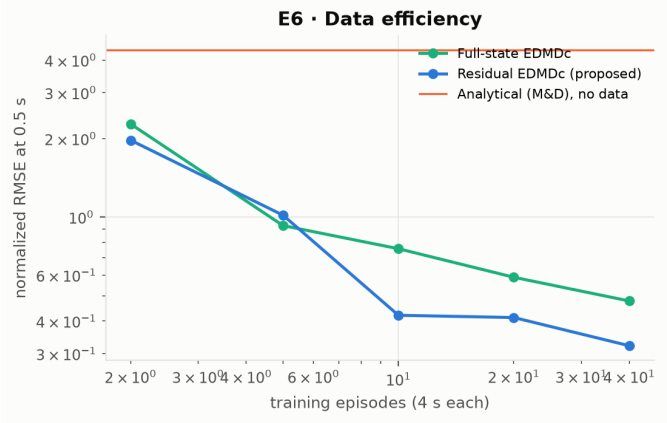


Figure 3: Prediction error at 0.5 s versus the number of training episodes (family A).

6.3 MPC in closed loop

Both MPCs use the analytical discrete model, a horizon of $N = 20$ steps of 5 ms, a terminal cost from the discrete Riccati equation, and thrust limits. They solve

$$\begin{aligned} \min_{\boldsymbol{u}_0,\dots,\boldsymbol{u}_{N-1}} \quad & \textstyle\sum_{j=0}^{N-1} \left(\boldsymbol{s}_j^\top Q \boldsymbol{s}_j + \boldsymbol{u}_j^\top R \boldsymbol{u}_j\right) + \boldsymbol{s}_N^\top P_N \boldsymbol{s}_N \\ \text{s.t.} \quad & \boldsymbol{s}_{j+1} = A_d \boldsymbol{s}_j + B_d \boldsymbol{u}_j + \hat{\boldsymbol{d}}, \quad f_{\min} - \bar{f}_i \le u_{j,i} \le f_{\max} - \bar{f}_i, \end{aligned} \tag{5}$$

with OSQP [40] through CVXPY [41]. The analytical MPC uses $\hat{\boldsymbol{d}} = 0$. The residual-Koopman MPC evaluates the learned residual at the current state outside the optimization and holds it constant over the horizon, as in [5], so the problem stays a convex QP. The solver itself needs about 0.7 ms per step; with the modelling layer the median wall-clock time is 4.4 ms, which leaves almost no margin to the 5 ms sample period. A real-time version would call the solver directly.

Table 6 breaks the steady altitude error down by mismatch effect. On family A the error is -0.29 m for LQR and the analytical MPC, of which the thrust loss alone accounts for -0.10 m and the inertia and motor mismatch for the rest. The residual-Koopman MPC reduces the error to -0.19 m (32% smaller), and it reduces it for each separate effect of family A. On the held-out family B the effects partly cancel; the residual-Koopman MPC reduces the error for the full plant and for its thrust loss alone, but increases it by 47% when the thrust loss is removed. As the controller that captures the vehicle after a detection delay (spin-up, then the controller, on family A), LQR recovers safely for delays up to 10 ms and both MPCs only without delay. The likely limiting factor is the validity of the linear prediction model far from the spin, rather than constraint handling.

Table 6: Steady altitude error near the relaxed hover (start 1 m below the set-point, mean over the last 2 of 8 s).

Plant	LQR	MPC, analytical	MPC, residual Koopman
A (full)	−0.285 m	−0.286 m	**−0.193** m
A without thrust loss	−0.154 m	−0.154 m	−0.066 m
Nominal + A's thrust loss only	−0.100 m	−0.101 m	−0.033 m
B (full, held out)	−0.029 m	−0.033 m	+0.022 m
B without thrust loss	+0.112 m	+0.111 m	+0.164 m
Nominal + B's thrust loss only	−0.174 m	−0.177 m	−0.107 m

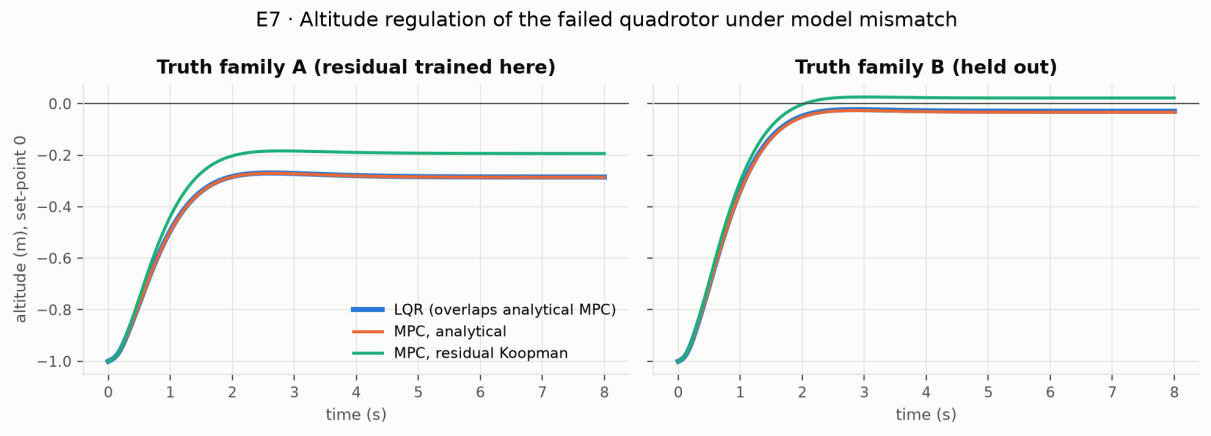


Figure 4: Altitude regulation of the failed vehicle under model mismatch.

7 Recovery under fault-detection delay

7.1 The undetected phase

At $t = 0$ rotor 4 fails. Until the failure is detected at $t = \tau$, the healthy LQR keeps running and its command to rotor 4 is lost. Near hover the body rates are small, so the gyroscopic terms in (2) vanish and the roll equation reduces to $I_{xx}^B \dot{p} = l(f_2 - f_4) = l f_2$. The healthy controller reacts by reducing f_2, but it keeps its motors running, so f_2 cannot go below the idle thrust $f_{\min}$. Therefore

$$\dot{p} \geq \frac{l f_{\min}}{I_{xx}^B} = 10.6 \text{ rad/s}^2 \tag{6}$$

for any healthy controller whose allocator keeps running motors at or above idle thrust, as standard autopilots do. Figure 5 confirms this: the tilt reaches about 13° at 100 ms and 37° at 200 ms, and three different tunings barely differ.

7.2 Effect of detection delay

We simulated hover, rotor failure at $t = 0$, the healthy LQR until $t = \tau$, and then the strategy of [1]: spin up on rotors 1 and 3 until $\|\boldsymbol{\omega}\| > 10$ rad/s, then the relaxed-hover LQR. The spin-up thrust is 2.95 N per rotor, read from the flight data in Fig. 4 of [1]; the hover-neutral value $mg/2 = 2.45$ N is a sensitivity case. The success criterion was fixed in code before the runs. A run is *safe* if the attitude is captured ($\|\boldsymbol{n} - \bar{\boldsymbol{n}}\| < 0.05$ and $\|\boldsymbol{\omega} - \bar{\boldsymbol{\omega}}\| < 2$ rad/s over the last second of a completed 10 s run), the altitude loss is below 2 m, and the vehicle never turns upside down. For each delay we simulate 40 random pre-failure states and report the fraction of safe runs with a 95% Wilson interval (Fig. 6, Table 7).

With the flight thrust and no delay, the vehicle first climbs 0.40 m and then drops only 0.03 m below its starting height, qualitatively like the flight, which climbs about 0.9 m. The probability

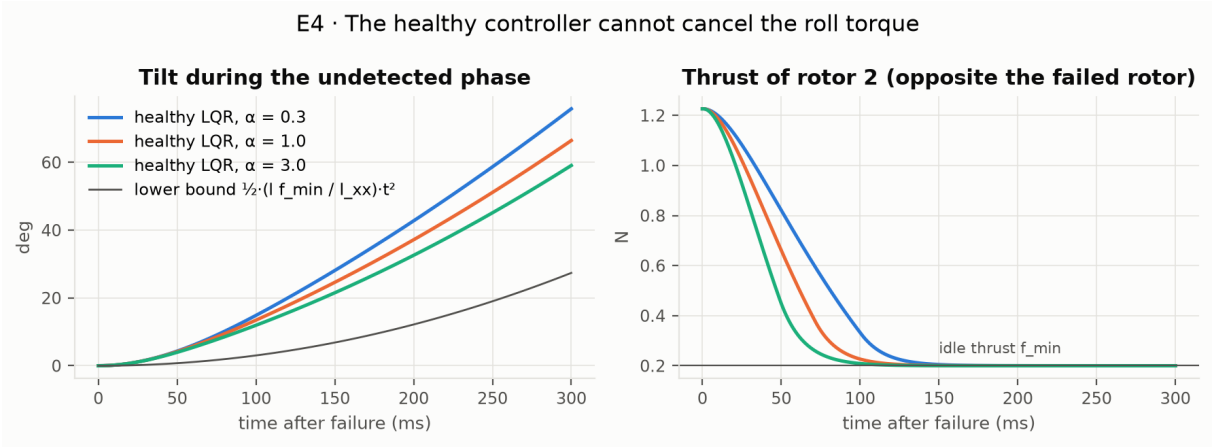


Figure 5: Undetected phase: tilt for three healthy-controller tunings (with the analytic lower bound) and the thrust of rotor 2, which falls to its idle limit.

of a safe recovery falls below 0.9 at about 12 ms of delay and below 0.5 at about 19 ms; with $mg/2$ the corresponding delays are 7 and 14 ms. If only attitude capture is required, P(safe) falls below 0.5 at 52 ms (40 ms with $mg/2$). The tolerance from the exact hover start is the same for three tunings of the healthy controller. Switching the LQR on directly, without spin-up, captures the attitude but turns the vehicle upside down and loses 9 to 16 m. Fast detectors need 30 to 130 ms [8], so with this strategy a safe recovery is unlikely within the detection time of current detectors. These numbers are specific to this strategy and model: nonlinear controllers have recovered with 0.1 to 0.18 s of detection delay [14, 25].

Table 7: Spin-up + LQR strategy: fraction of safe recoveries over 40 random pre-failure states (velocity $\sigma = 0.3$ m/s, angles $\sigma = 3°$, rates $\sigma = 0.3$ rad/s), with 95% Wilson intervals.

Delay τ	0 ms	10 ms	15 ms	20 ms	30 ms	40 ms
2.95 N	1.00 [0.91, 1]	0.97 [0.87, 1]	0.80 [0.65, 0.90]	0.42 [0.29, 0.58]	0.17 [0.09, 0.32]	0.07 [0.03, 0.20]
2.45 N	1.00 [0.91, 1]	0.82 [0.68, 0.91]	0.45 [0.31, 0.60]	0.28 [0.16, 0.43]	0.07 [0.03, 0.20]	0.07 [0.03, 0.20]

7.3 A first saturation-aware certificate

For the closed loop $\dot{\boldsymbol{x}} = A\boldsymbol{x} + B\,\mathrm{sat}(F\boldsymbol{x})$ with $F = -K$ and symmetric saturation levels $\bar{\boldsymbol{u}}$, the ellipsoid $\mathcal{E} = \{\boldsymbol{x} : \boldsymbol{x}^\top Q^{-1}\boldsymbol{x} \le 1\}$ is contractively invariant if there is a matrix $Y = HQ$ such that [42, 43]

$$AQ + QA^\top + B(DFQ + D^-Y) + (DFQ + D^-Y)^\top B^\top + 2\alpha Q \preceq -\epsilon I \quad \forall D \in \mathcal{D}, \tag{7}$$

$$\begin{bmatrix} \bar{u}_i^2 & Y_i \\ Y_i^\top & Q \end{bmatrix} \succeq 0, \quad i = 1, \ldots, m, \tag{8}$$

where $\mathcal{D}$ is the set of diagonal 0/1 matrices, $D^- = I - D$, and α is a required decay rate. The symmetric levels are the smaller distance from the trim to either thrust limit, so the certificate also holds for the plant's asymmetric limits. We maximize the extent of $\mathcal{E}$ along the direction from the equilibrium to the state right after a failure in hover. The motor states are bounded by their physical range; without this bound the problem is ill-conditioned and certifies a physically meaningless set. The problems were solved with CVXPY [41] and Clarabel [44], with SCS [45] as a fallback.

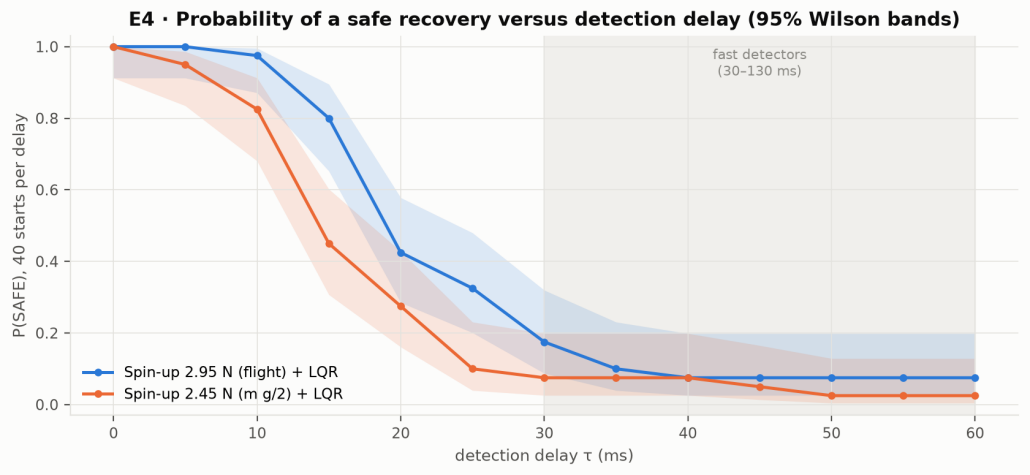


Figure 6: Probability of a safe recovery versus detection delay for the two spin-up thrusts, with 95% Wilson bands. The shaded region marks the delays of fast detectors [8].

The certificate covers $\beta = 0.22$ of the way to the hover-onset state. In nonlinear simulation, every start up to $\beta = 0.7$ recovers without inverting, so the certificate is about three times conservative in this direction. This LQR does not recover safely from the hover-onset state itself. On a two-dimensional slice (Fig. 7), all failures occur at spin rates of 4.8 rad/s or less, and none at 6.8 rad/s or more. A certificate built from the linearization at the full spin cannot reach that region. The next step is therefore a certificate valid over the whole spin-up, treating the yaw rate r as a varying parameter. LPV synthesis on the yaw rate [17] and recoverable sets at the relaxed hover [23] exist; our aim is to use such a set as the capture target of a detection-delay budget.

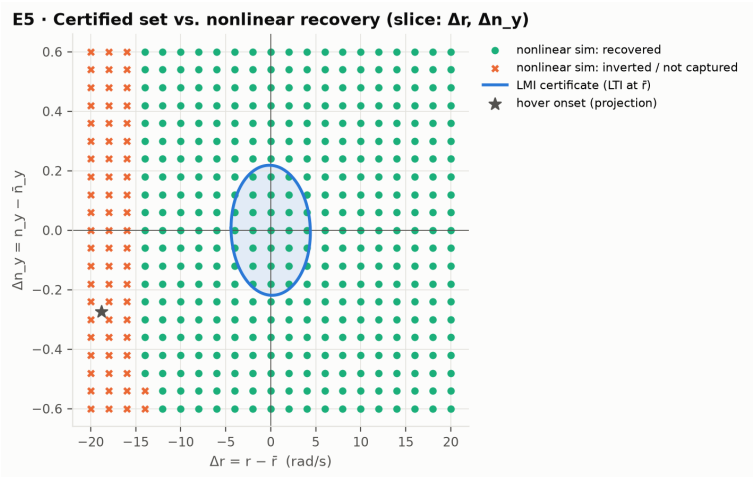


Figure 7: Slice $(\Delta r, \Delta n_y)$ of the certified ellipsoid (decay rate $0.1\,\mathrm{s}^{-1}$) against a grid of nonlinear simulations. 376 of 441 grid points recover without inverting; 7% of these lie inside the certified slice.

8 Limitations

1. **Spin-up thrust.** The delay tolerance depends on the spin-up thrust (12 against 7 ms for the 90% level). The flight value is read from a plot, so it carries some uncertainty.

2. **Nominal plant for the delay study.** The delay study uses the nominal plant; the papers' models over-predict the steady spin rate in flight by about 20%.

3. **Generalization of the learned models.** The residual model helps reliably only on the plant family it was trained on.

4. **Linear certificates.** The certificate holds for the linearized saturated loop. The gap to the nonlinear system is measured by simulation, not proven.

5. **Truth-plant realism.** The truth plants add thrust loss, inertia error and motor lag, but not yet air-relative thrust or quadratic drag, both of which are validated in our implementation of the IJRR model.

6. **Certificate for the learned controller.** The clipping level $\boldsymbol{d}_{\max}$ is a percentile of the training residuals, not a bound on the true mismatch, and a constrained MPC needs more than the saturation conditions above. Both must be addressed before the residual-Koopman loop can be certified.

7. **Literature access.** Several closely related papers [11, 16, 17, 23] have not yet been read in full; our novelty statements follow from abstracts, theses and citing papers.

9 Plan for the remaining weeks

The course has a final progress presentation on 31 October 2026, and the journal-style final report is due on 13 November 2026. The remaining work is planned as follows.

Dates	Work
12 to 18 Oct	LPV model in the yaw rate and a saturation-aware certificate over the spin-up. MPC on the full-state Koopman model (the second controller of our proposal) and an integral-action MPC baseline. Read the remaining key papers in full.
19 to 25 Oct	Reachable set of the undetected phase and the containment test, giving a first certified $\tau^\star$; comparison with the empirical P(safe) curve. Online adaptation of the residual by recursive least squares.
26 to 31 Oct	Sensitivity of $\tau^\star$ to the spin-up thrust, idle thrust and motor lag; air-relative thrust in the truth plant. Final progress presentation on 31 Oct.
1 to 13 Nov	Bound on the true model mismatch and a certificate for the residual-Koopman loop if time allows; final simulations and the journal-style report, due 13 Nov.

The main risk is that a single quadratic Lyapunov function over the whole spin-up is infeasible or too conservative. The fallbacks are a parameter-dependent Lyapunov function, or a chain of nested ellipsoids along the spin-up.

10 Conclusion

We built and validated a simulator of a quadrotor with complete rotor loss. On the training plant, the residual Koopman model is the most accurate predictor from 100 ms on, needs less data than a full-state Koopman model, and reduces a steady altitude error by one third inside a convex MPC; it does not yet generalize to an unseen plant. In our simulation, the published recovery strategy recovers safely with high probability only for detection delays of about 10 to 20 ms, well below the 30 to 130 ms that fast detectors need. A first LMI certificate covers 22% of the way to the hover-onset state, and the controller itself fails only at low spin rates, which points to a certificate over the spin-up as the next step. The remaining weeks build that certificate and the certified delay budget, and evaluate each capture controller, including the residual-Koopman MPC, by the delay it tolerates.

A Software and reproducibility

The simulator, analysis and experiments are written in Python with NumPy [46], SciPy [47], the Python Control Systems Library [48] and Matplotlib [49]. Convex problems are modelled with CVXPY [41] and solved with Clarabel [44], SCS [45] and OSQP [40]; tests use pytest [50]. All random seeds are fixed, and every table and figure is produced by one script in `experiments/`. The code is available at https://github.com/pranshuk22/quadrotor-rotor-loss-recovery.

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python -m pytest -q
.venv/bin/python experiments/v0_validation_report.py
.venv/bin/python experiments/e1_equilibrium.py   # and e2 to e7
```

References

[1] Mark W. Mueller and Raffaello D'Andrea. Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers. In *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, pages 45–52, 2014.

[2] Mark W. Mueller and Raffaello D'Andrea. Relaxed hover solutions for multicopters: Application to algorithmic redundancy and novel vehicles. *The International Journal of Robotics Research*, 35(8):873–889, 2016. doi: 10.1177/0278364915596233.

[3] Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018.

[4] Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25:1307–1346, 2015.

[5] Edoardo Todde, Simone Martini, Alessandro Rizzo, and Kimon P. Valavanis. Residual Koopman-based model predictive control of quadrotors. In *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 50–56, 2026. doi: 10.1109/ICUAS69441.2026.11598621.

[6] D. Bruder, D. Bombara, and R. J. Wood. A Koopman-based residual modeling approach for the control of a soft robot arm. *The International Journal of Robotics Research*, 2025.

[7] Narayanan and Vaidya. RK-MPC: Residual Koopman model predictive control for quadruped locomotion in offroad environments, 2026. arXiv:2604.04221.

[8] Bram Strack van Schijndel, Sihao Sun, and Coen de Visser. Fast fault detection on a quadrotor using onboard sensors and a Kalman filter approach, 2021. arXiv:2102.06439.

[9] Simha, Vadgama, and Raha. A geometric approach to rotor failure tolerant trajectory tracking control design for a quadrotor, 2017. arXiv:1704.00327.

[10] Mohamed Al Lawati and Adeel Akhtar. Quasi-static fault-tolerant feedback control of a quadrotor under rotor failure with provable safety guarantees, 2026. arXiv:2609.23918.

[11] Hasan Basak and Emmanuel Prempain. Switching recovery control of a quadcopter UAV. In *Proc. European Control Conference (ECC)*, pages 3641–3646, 2015. doi: 10.1109/ECC.2015.7331096.

[12] Hasan Basak. *Robust switching recovery control of a quadcopter aerial vehicle model*. PhD thesis, University of Leicester, 2018.

[13] Chenxu Ke, Kai-Yuan Cai, and Quan Quan. Uniform passive fault-tolerant control of a quadcopter with one, two, or three rotor failure. *IEEE Transactions on Robotics*, 39(6): 4297–4311, 2023. doi: 10.1109/TRO.2023.3297048.

[14] Jiehao Chen, Kaidong Zhao, Zihan Liu, YanJie Li, and Yunjiang Lou. Learning-based passive fault-tolerant control of a quadrotor with rotor failure, 2025. arXiv:2503.02649.

[15] Alessandro Freddi, Alexander Lanzon, and Sauro Longhi. A feedback linearization approach to fault tolerance in quadrotor vehicles. In *Proc. 18th IFAC World Congress*, pages 5413–5418, 2011.

[16] Alexander Lanzon, Alessandro Freddi, and Sauro Longhi. Flight control of a quadrotor vehicle subsequent to a rotor failure. *Journal of Guidance, Control, and Dynamics*, 37(2), 2014.

[17] Johannes Stephan, Lorenz Schmitt, and Walter Fichter. Linear parameter-varying control for quadrotors in case of complete actuator loss. *Journal of Guidance, Control, and Dynamics*, 41(10):2232–2246, 2018. doi: 10.2514/1.G003441.

[18] Ahmed Khattab, Halim Alwi, and Christopher Edwards. Mitigating total rotor failure in quadrotor using LPV based sliding mode control scheme. In *Proc. 4th Conference on Control and Fault Tolerant Systems (SysTol)*, pages 98–103, 2019. doi: 10.1109/SYSTOL.2019.8864740.

[19] Sihao Sun, Xuerui Wang, Qiping Chu, and Coen de Visser. Incremental nonlinear fault-tolerant control of a quadrotor with complete loss of two opposing rotors. *IEEE Transactions on Robotics*, 2020. doi: 10.1109/TRO.2020.3010626. arXiv:2002.07837.

[20] Sihao Sun, Matthias Baert, Bram Strack van Schijndel, and Coen de Visser. Upset recovery control for quadrotors subjected to a complete rotor failure from large initial disturbances, 2020. arXiv:2002.09425.

[21] Fang Nan, Sihao Sun, Philipp Foehn, and Davide Scaramuzza. Nonlinear MPC for quadrotor fault-tolerant control. *IEEE Robotics and Automation Letters*, 2022. arXiv:2109.12886.

[22] Zhiwen Sun, Fubiao Zhang, and Shuaipeng Lang. Fault-tolerant model predictive control of a quadrotor with an unknown complete rotor failure. In *Proc. 41st Chinese Control Conference (CCC)*, pages 4087–4093, 2022. doi: 10.23919/CCC55666.2022.9901659.

[23] Xin Qi, Didier Theilliol, Yuqing He, and Jianda Han. Recoverable set computation for post-fault/failure quadrotors based on sum of squares (SOS). In *Proc. 3rd Conference on Control and Fault-Tolerant Systems (SysTol)*, pages 73–78, 2016. doi: 10.1109/SYSTOL.2016.7739731.

[24] Eeshan Kulkarni, Narasimhan Sundararajan, and Suresh Sundaram. Heli-quad design for full-attitude fault-tolerant control under complete failure of an actuator, 2024. arXiv:2011.07529v3.

[25] Xiaobin Zhou, Miao Wang, Chengao Li, Can Cui, Ruibin Zhang, Yongchao Wang, Chao Xu, and Fei Gao. Rotor-failure-aware quadrotors flight in unknown environments, 2025. arXiv:2510.11306.

[26] Chengyu Yang, Jiahao Yu, Sheng Cheng, Ziyin Han, Junjie Gao, Bihao Mo, and Naira Hovakimyan. Failure detection and recovery for quadrotors in the presence of severe rotor failures with multiple model $\mathcal{L}_1$ adaptive controller. *IEEE Robotics and Automation Letters*, 11(1):346–353, 2026. doi: 10.1109/LRA.2025.3630964.

[27] Florin Stoican and Sorin Olaru. *Set-Theoretic Fault-Tolerant Control in Multisensor Systems*. Wiley-ISTE, 2013.

[28] Jong-Yeob Shin and Christine M. Belcastro. Performance analysis on fault tolerant control system. *IEEE Transactions on Control Systems Technology*, 14(5):920–925, 2006.

[29] Youngim Nam, Kangmin Lee, Hyo-Sang Shin, and Cheolhyeon Kwon. Fault tolerant motion planning for a quadrotor subject to complete rotor failure. *Aerospace Science and Technology*, 154:109529, 2024. doi: 10.1016/j.ast.2024.109529.

[30] Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman invariant subspaces and finite linear representations of nonlinear dynamical systems for control. *PLoS ONE*, 11(2):e0150171, 2016.

[31] S. S. Narayanan, D. Tellez-Castro, S. Sutavani, and U. Vaidya. SE(3) Koopman-MPC: Data-driven learning and control of quadrotor UAVs. *IFAC-PapersOnLine*, 56(3):607–612, 2023.

[32] S. Martini, E. Todde, M. Stefanovic, M. J. Rutherford, A. Rizzo, and K. P. Valavanis. Koopman-based model predictive control of quadrotors. In *Proc. International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 207–213, 2025.

[33] Jayden Dongwoo Lee, Hyeonmin Kim, Yoonseong Kim, and Hyochoong Bang. Koopman fault-tolerant model predictive control for multirotor with actuator fault. In *Proc. 24th International Conference on Control, Automation and Systems (ICCAS)*, pages 231–236, 2024. doi: 10.23919/ICCAS63016.2024.10773232.

[34] Jayden Dongwoo Lee, Sukjae Im, Lamsu Kim, Hyungjoo Ahn, and Hyochoong Bang. Data-driven fault detection and isolation for multirotor system using Koopman operator. *Journal of Intelligent & Robotic Systems*, 110(3):128, 2024. doi: 10.1007/s10846-024-02142-y.

[35] Yonghao Fu, Cheng Hu, Haokun Xiong, Zhanpeng Bao, Wenyuan Du, Edoardo Ghignone, Michele Magno, Lei Xie, and Hongye Su. Residual Koopman model predictive control for enhanced vehicle dynamics with small on-track data input, 2025. arXiv:2507.18396.

[36] Ao Jin, Qiujin Liang, Tao Zhang, Panfeng Huang, and Fan Zhang. Differentiable physics-informed adaptive Koopman control for stable flight under unknown disturbances, 2025. arXiv:2506.08319.

[37] Robin Strässer, Manuel Schaller, Karl Worthmann, Julian Berberich, and Frank Allgöwer. SafEDMD: A Koopman-based data-driven controller design framework for nonlinear dynamical systems, 2024. arXiv:2402.03145.

[38] Sihao Sun, Giovanni Cioffi, Coen de Visser, and Davide Scaramuzza. Autonomous quadrotor flight despite rotor failure with onboard vision sensors: Frames vs. events. *IEEE Robotics and Automation Letters*, 2021. arXiv:2102.13406.

[39] Prashant Solanki and Coen C. de Visser. Attitude estimation of a quadcopter with one fully damaged rotor using on-board MARG sensors. In *AIAA SciTech 2022 Forum*, 2022. doi: 10.2514/6.2022-0857.

[40] Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: An operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020.

[41] Steven Diamond and Stephen Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016.

[42] Tingshu Hu and Zongli Lin. *Control Systems with Actuator Saturation: Analysis and Design*. Birkhäuser, Boston, 2001.

[43] Stephen Boyd, Laurent El Ghaoui, Eric Feron, and Venkataramanan Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*. SIAM, Philadelphia, 1994.

[44] Paul J. Goulart and Yuwen Chen. Clarabel: An interior-point solver for conic programs with quadratic objectives, 2024. arXiv:2405.12762.

[45] Brendan O'Donoghue, Eric Chu, Neal Parikh, and Stephen Boyd. Conic optimization via operator splitting and homogeneous self-dual embedding. *Journal of Optimization Theory and Applications*, 169(3):1042–1068, 2016. doi: 10.1007/s10957-016-0892-3.

[46] Charles R. Harris et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[47] Pauli Virtanen et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020. doi: 10.1038/s41592-019-0686-2.

[48] Sawyer Fuller, Ben Greiner, Jason Moore, Richard Murray, René van Paassen, and Rory Yorke. The Python Control Systems Library (python-control). In *Proc. 60th IEEE Conference on Decision and Control (CDC)*, pages 4875–4881, 2021. doi: 10.1109/CDC45484.2021.9683368.

[49] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.

[50] Holger Krekel et al. pytest. `https://pytest.org`, 2026. Version 9.1.